

LangGraph Chatbot on Groq — Project Explanation

Overview

This project, the eighth entry in my Agentic AI series, builds the smallest complete LangGraph application: a terminal chatbot made of a single graph node, powered by Groq's fast LPU inference running the open-weights gpt-oss-20b model. It deliberately switches away from OpenAI to show that a LangGraph application is independent of the model provider behind it.

What the Notebook Does

The Groq API key is collected at runtime with `getpass` and the model is loaded through LangChain's `ChatGroq` class. A typed state with a single `messages` field uses the `add_messages` reducer, so a node only returns its new message and LangGraph appends it to the conversation instead of replacing it. A single chatbot node sends the current messages to the model and returns the reply, and the graph is wired from `START` to the chatbot node to `END`, compiled, and rendered as a Mermaid diagram to confirm the structure visually.

Responses are produced with `graph.stream` rather than `graph.invoke`, which yields an update for each node as it runs; the notebook prints only messages generated by the AI. An interactive loop keeps the conversation running until the user types one of six exit keywords, and wraps each turn in error handling so a failed API call stops the loop with a readable message. The accompanying README also documents an important limitation: the graph is compiled without a checkpoint, so each turn starts with fresh state and the bot does not remember earlier messages, with LangGraph's `MemorySaver` and a thread identifier shown as the fix.

Skills Demonstrated

- Building a minimal LangGraph state graph with a message reducer
- Streaming graph output for responsive chat interfaces
- Using Groq inference and an open-weights model through LangChain
- Recognising the difference between in-run message merging and cross-turn memory

Provider-independent graph design is directly relevant to my dissertation's agentic intelligence platform, which needs to switch between hosted and open-weights models without changing its orchestration layer.